DFS 시간 복잡도 & 응용 p.14-16

시간 복잡도 — 모든 정점은 정확히 *한 번*만 방문. 인접 탐색 비용이 자료구조에 따라 다름.

리스트: O(n+e)
행렬: O(n²)

- **인접 리스트**: 정점 방문 시 O(degree) → aggregate: T = c·n + e → **O(n+e)**
- **인접 행렬**: 정점 방문 시 O(n) → T = n·(c₁+c₂·n) → **O(n²)**
- sparse 그래프에서는 인접 리스트가 압도적으로 유리

DFS 응용 — DFS 한 번으로 여러 구조적 정보를 동시에 얻는다.

- **Spanning Tree**: unvisited 정점 도달 시 사용한 간선들 → G가 connected이면 n−1개 간선
- **Path Finding**: v에서 DFS 시작, w 방문되거나 unvisited 소진 시 종료
- **Connected 판별**: 임의 정점 DFS 후 모든 정점 방문됐으면 connected
- **Connected Components**: unvisited 정점마다 DFS 재시작 → 각 호출 = 한 component

Strong Components — Kosaraju 알고리즘 p.17-20

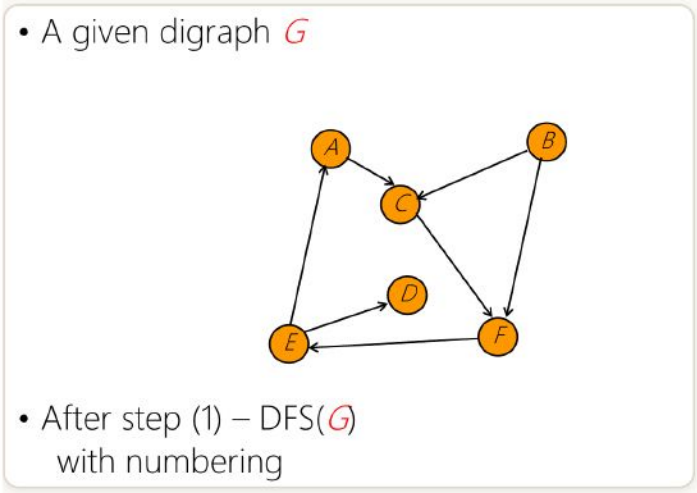
Strong Component — 유향 그래프에서 **서로 도달 가능한** 정점의 최대 집합.

- 입력: digraph G / 출력: strong components 집합

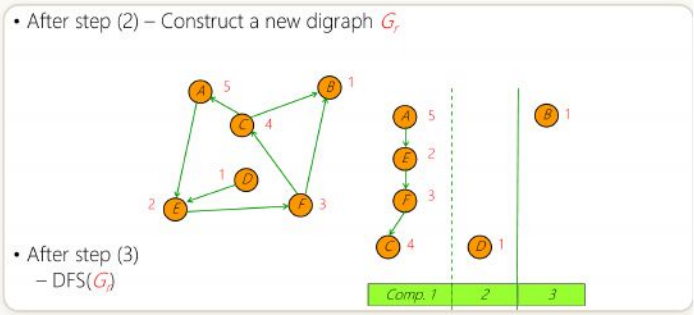
Kosaraju — Strong Components

① **DFS(G):** 재귀 완료 순서로 각 정점에 번호 부여
   (완료 순서 = **postorder** — 방문 순서와 다름!)
② G의 모든 **arc** 방향 뒤집기 → **Gr** 생성
③ **DFS(Gr):** ①의 번호가 가장 큰 정점부터 시작
   → 각 **DFS** 호출이 도달하는 정점들 = 하나의 **strong component**

복잡도: 두 번의 DFS = O(n+e). 완료 순서가 sink를 먼저 가리키게 만드는 것이 핵심.



Step①: G의 DFS — 완료번호 부여 (p.18)



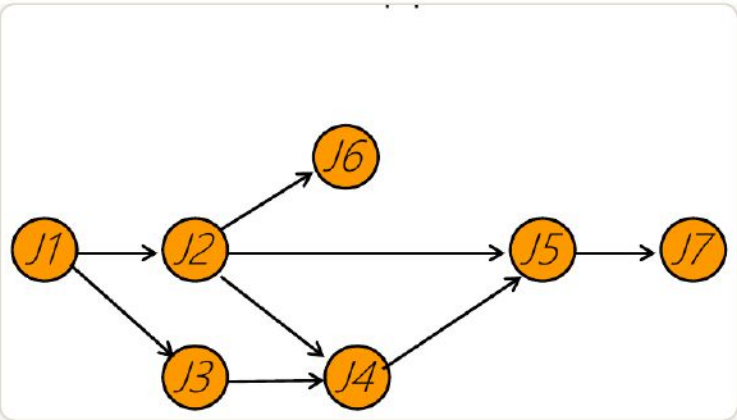
Step②③: Gr + DFS → 3개 성분 (p.19)

⚠ **함정** 완료 순서(*postorder*)와 방문 순서를 혼동하지 말 것. 완료 번호가 **큰** 정점 = DFS에서 *가장 늦게 완료* = sink 쪽. Gr에서는 이 정점에서 출발해야 source 쪽을 묶을 수 있다.

Topological Sort — 위상 정렬 p.21-24

Topological Sort — DAG 정점의 **선형 순서**: path v→w가 있으면 v가 w보다 앞에 온다.

- 전제 조건: **DAG (Directed Acyclic Graph)** 에만 정의됨
- valid ordering은 일반적으로 여러 개 존재 (unique하지 않음)



예제 DAG: J1→J2→J5→J7 등 다수의 valid ordering 존재 (p.21)

방법 1 — DFS 기반 (Stack) — DFS 수행 시 정점 *완료 시점*에 print → 결과는 topo sort의 **역순**.

- 방문 시점: 아무 동작 없음 / 완료(pop back) 시점: print
- 역순을 뒤집으면 유효한 topological ordering

TOPSORT(v) — DFS 기반

```
mark[v] := visited;
for (each unvisited vertex w adjacent from v)
  TOPSORT(w);
print(v);    ← 완료 시점에 출력 (역순 → 뒤집으면 topo order)
```

방법 2 — Queue 기반 (in-degree) — in-degree 0인 정점을 enqueue → dequeue & 출력 → 나가는 간선 제거 → 반복. 결과가 **정방향**.

큐: 정방향
DFS: 역순

- in-degree 0 정점이 여럿이면 어느 것을 먼저 골라도 valid ordering
- **Cycle 검출**: 처리 후 남은 정점 수 > 0 이면 → **cycle 존재** 판정

방식	자료구조	출력 순서	cycle 검출
DFS 기반	recursion stack	역순 → 뒤집기	별도 처리 필요
큐(in-degree)	FIFO queue	정방향	**처리 후 미처리 정점 > 0**

두 방식 비교. 큐 기반이 cycle 검출에도 직접 활용 가능.

Breadth-First Search (BFS) p.25-34

BFS 정의 — 같은 depth의 정점을 **가능한 한 넓게** 모두 방문 후 다음 depth로. level-order 탐색.

- 구현: *iterative* — DFS 재귀 스택 대신 **FIFO queue** 사용
- 스택 오버플로 없음, 깊이 제한 없음

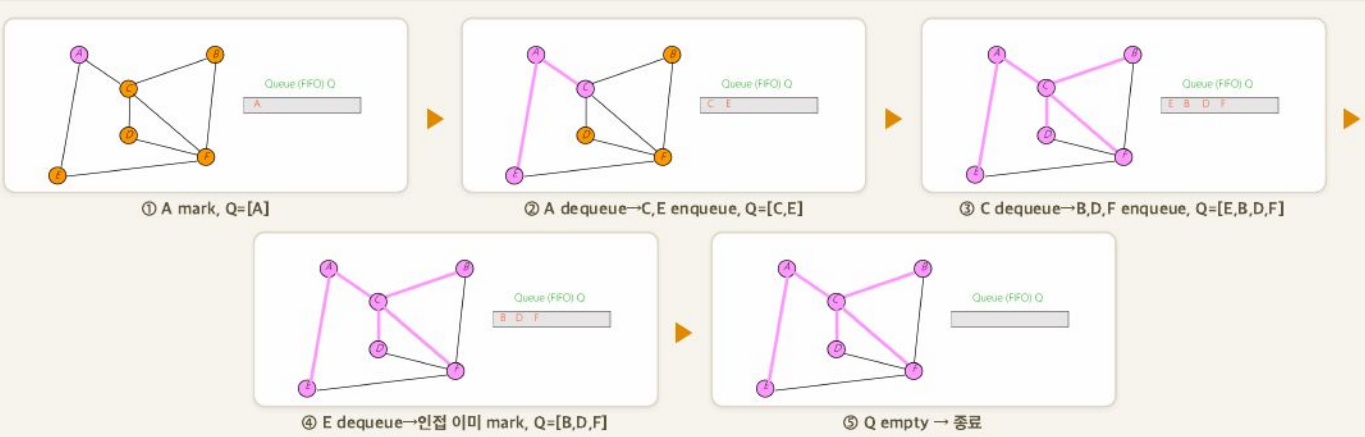
BFS

```
Visit/Mark start vertex & enqueue;
repeat until queue is empty:
  dequeue vertex v;
  for (each unvisited vertex w adjacent from v):
    Visit/Mark w;
    enqueue w;
```

dequeue 순서 = 방문 순서 = 거리 순서(source로부터 간선 수 기준 오름차순).

BFS 전체 과정 예제 — Q=[A] → A dequeue, C·E enqueue → C dequeue, B·D·F enqueue → E·B·D·F 처리 → Q empty.

- 방문 순서: A → C, E (depth 1) → B, D, F (depth 2)
- 처음 mark되는 시점의 depth = **source로부터의 최단 거리(간선 수)**



① A mark, Q=[A] ② A dequeue→C,E enqueue, Q=[C,E] ③ C dequeue→B,D,F enqueue, Q=[E,B,D,F]

④ E dequeue→인접 이미 mark, Q=[B,D,F] ⑤ Q empty → 종료

BFS 전체 과정 — FIFO가 곧 level-order 보장 (p.26-33)

BFS 성질 & 응용 — DFS와 동일한 **O(n+e)** 복잡도(인접 리스트). 동일하게 적용 가능한 성질:

- Connectivity 판별 / Connected components 식별 / Spanning tree / Path finding
- **unweighted 최단 경로**: BFS가 **유일한 표준**. level-order = 거리 순서 → 처음 도달 = 최단
- DFS는 첫 경로가 최단이라는 보장 없음 → unweighted 최단경로에 *부적합*

DFS vs BFS 비교 p.35-36

	DFS	BFS
자료구조	recursion stack (암시적)	**explicit FIFO queue**
탐색 방향	깊이 우선 (depth-first)	너비 우선 (breadth-first)
시간복잡도	**O(n+e) 리스트 / O(n²) 행렬**	**O(n+e) 리스트 / O(n²) 행렬**
spanning tree	DFS spanning forest	BFS spanning tree
unweighted 최단경로	**불가 (경로 비최단 가능)**	**BFS 우위 (level=거리)**
응용 특화	strong components, topo sort	최단경로(unweighted)

두 알고리즘의 자료구조 선택 하나가 탐색 성격과 응용 범위를 결정.

✎ **핵심** mark bit + **stack(DFS)** = 깊이 우선·재귀·backtrack·spanning forest·strong components·topo sort. mark bit + **queue(BFS)** = 너비 우선·iterative·level-order·unweighted 최단경로. 복잡도는 **인접 리스트 기준 둘 다 O(n+e)** — 인접 행렬이면 O(n²).

★ **필수암기** DFS=재귀스택·O(n+e)/O(n²)·spanning forest·component수=트리수·strong comp(Kosaraju 2×DFS+Gr)·topo sort(완료시점print역순)·BFS=FIFO큐·동일복잡도·level-order·처음mark=최단거리(unweighted)·cycle→topo sort 불가·큐기반 미처리정점>0이면cycle

Ch 15 Shortest Path — 최단 경로

가중 유향 그래프에서 비용 합이 최소인 경로 찾기 · Dijkstra · Floyd

문제 정의 & 4가지 유형 p.1-8

SPP (Shortest Path Problem) — *비음(non-negative)* 가중 유향 그래프에서 두 정점 사이 **간선 가중치 합이 최소**인 경로를 찾는 문제. path length = 경로 위 모든 가중치의 합.

- **Single-pair** — source 1개 · destination 1개
- **Single-source** — source 1개 · **모든** destination ⟶ **Dijkstra**
- **Single-destination** — 간선 뒤집으면 single-source로 환원
- **All-pairs** — 모든 쌍 ⟶ **Floyd**

Naive Greedy의 실패 — "도달 가능한 정점 중 *최저 가중 간선*으로 다음 정점 선택"하는 단순 greedy는 전역 최적을 보장 못 함.

- A→E 예시: greedy는 길이 21을 내지만 실제 최단은 18 ⟶ *Algorithm doesn't work*
- Dijkstra는 같은 greedy지만 D[v]의 정의가 달라서 동작 (아래)

Dijkstra Algorithm

single-source · 비음 가중치

상태 두 배열 `D[i]` · `P[i]` — `D[i]` = source에서 i까지의 *현재* 최단거리 (= S 안의 정점만 통과하는 최단경로 길이). `P[i]` = i 직전 predecessor → 역추적으로 실제 경로 복원.

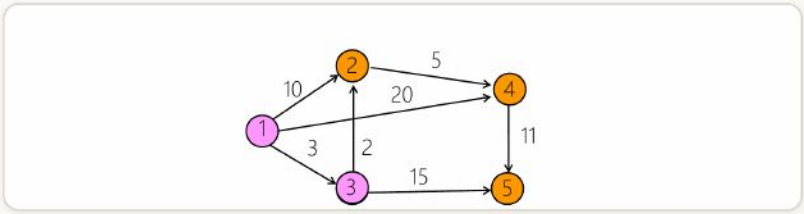
D=거리, P=직전점
S=확정집합

- `S` = source로부터 최단거리가 *이미 확정된* 정점 집합
- 매 stage: `V−S` 중 D[.] 최소인 `w` 를 S에 추가 → 그 순간 D[w]가 곧 최단거리

Dijkstra (경로까지 기록)

```
Initialize:
  S := {1}
  for i := 2 to n do
    D[i] := C[1,i]
    if C[1,i] ≠ ∞ then P[i] := 1     ⟵ source를 직전점으로
for stage := 1 to n−1 do
  w := argmin D[v] for v in V−S      ⟵ one-edge extension
  S := S ∪ {w}
  for each v in V−S do               ⟵ relaxation
    if D[w] + C[w,v] < D[v] then
      D[v] := D[w] + C[w,v]
      P[v] := w
```

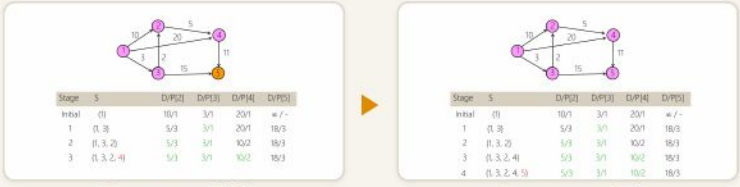
relaxation = 한-간선 이완. D[w]+C[w,v]가 기존 D[v]보다 작으면 갱신.



예제 그래프 — source=1 (p.13)



① Initial — S={1}　② Stage 1 — 3 확정　③ Stage 2 — 2 확정



④ Stage 3 — 4 확정　⑤ Stage 4 — 5 확정

Dijkstra 전체 진행 — 빨간 화살표=relax된 간선, 녹색=확정 (p.13-17)

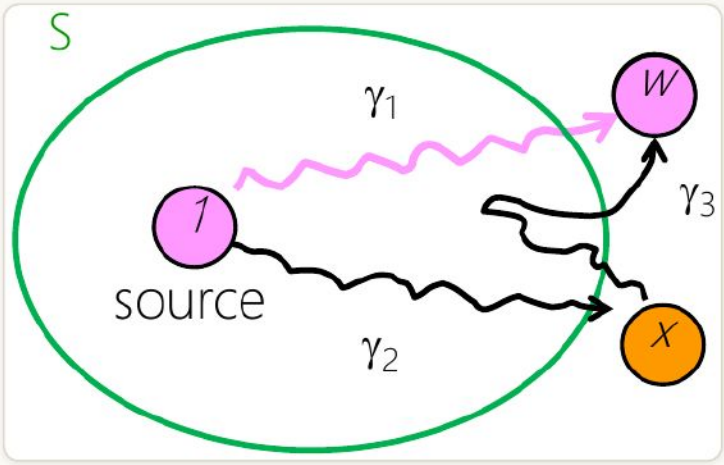
Progress 추적 — 전체 과정 — stage마다 D[.] 최소 정점을 확정(S에 추가)하고 이웃을 relax. 거리/직전점을 D/P로 표기.

Stage	S	D/P[2]	D/P[3]	D/P[4]	D/P[5]
Initial	{1}	10/1	3/1	20/1	∞ / -
1	{1,3}	5/3	3/1	20/1	18/3
2	{1,3,2}	5/3	3/1	10/2	18/3
3	{1,3,2,4}	5/3	3/1	10/2	18/3
4	{1,3,2,4,5}	5/3	3/1	10/2	18/3

확정(초록)·relax로 갱신된 값(분홍). 최단거리 D[5]=18은 1→3→5(3+15).

정당성 — Greedy Choice Property — 매 단계 *locally-best*(D 최소) 선택이 *전역 최선*임을 **귀류법**으로 증명.

- 가정: S 밖 정점 x 경유 경로 $|\gamma_2|+|\gamma_3| < |\gamma_1|$(현재 D[w])
- 그러면 $|\gamma_2| < |\gamma_1|$ → **x가 w보다 먼저 뽑혔어야** 함 → **모순**



S 밖 x 경유가 더 짧다면 모순 (p.21)

> ⚠ **함정** 이 증명은 **비음 가중치** 가정에 의존. **음수 간선**이면 x→w가 음수일 때 $|\gamma_2|+|\gamma_3|<|\gamma_1|$이어도 $|\gamma_2|<|\gamma_1|$이 깨짐 ⟶ Dijkstra가 *틀린 최단거리*를 보고할 수 있다.

복잡도 — 자료구조에 따라 둘 — 병목은 "*D 최소 정점 선택*" 연산.

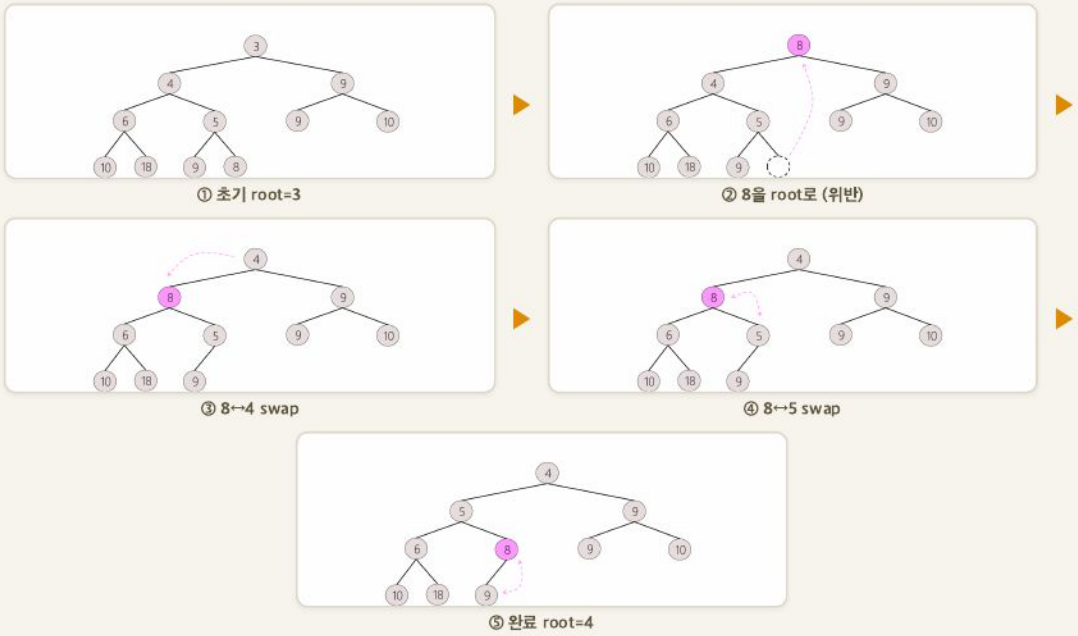
- **Option-1** 전체 리스트 스캔: 선택 $O(n^2)$ + 갱신 $O(e)$ = **$O(n^2)$** → *밀집 그래프* 유리
- **Option-2** min-heap: DeleteMin n번 $O(n \log n)$ + 갱신 e번 $O(e \log n)$ = **$O(e \log n)$** → *희소 그래프* 유리

Min-Heap DeleteMin

Option-2의 핵심 연산 · O(log n)

DeleteMin — min-heap(부모≤자식, 완전이진트리)에서 **최소(root)**를 빼서 반환. 2단계로 동작.

- **Step-1** root 제거 → *마지막 leaf*를 root 자리(hole)로 이동
- **Step-2** **down-heap bubbling** — 더 작은 자식과 비교·swap을 leaf까지 반복



down-heap bubbling 전체 과정 — 더 작은 자식과 swap 반복 (p.25-33)

> ✎ **핵심** down-heap에서 **두 자식 중 더 작은 값**과 비교해야 swap 후에도 부모≤두자식이 유지된다. 비용 ∝ height = **O(log n)** → Dijkstra heap 버전 O((n+e)log n)이 여기서 나옴.

Dijkstra 확장·한계

- **Cyclic graph** — **OK**. update rule이 우회경로를 즉시 갱신
- **Negative edge** — **실패**. greedy choice 증명이 무너짐
- **DAG** — **topological sort** 순서로 relaxation 한 패스면 충분(더 빠름)

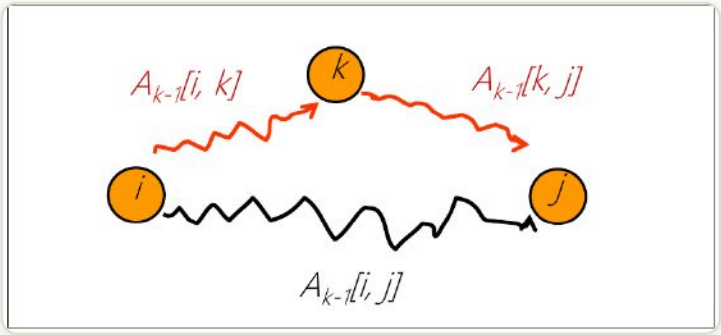
Floyd Algorithm

all-pairs · DP · $O(n^3)$

DP 정의 $A_k[i,j]$ — i→j 경로 중 **중간 정점 인덱스가 모두 ≤ k**인 것의 최단 길이. k를 0→n까지 키우며 k번째 정점을 차례로 *중간 경유 허용*.

중간점 ≤ k 허용하며 확장

- 점화식: $A_k[i,j] = \min\{ A_{k-1}[i,j],\ A_{k-1}[i,k]+A_{k-1}[k,j] \}$
- k 경유 vs 직접을 비교 — 모든 (i,j) 쌍에 적용



k 경유 vs 직접 (p.39)

Floyd + 경로기록(Next)

```
for k := 1 to n do
  for i := 1 to n do
    for j := 1 to n do
      if A[i][k] + A[k][j] < A[i][j] then
        A[i][j]    := A[i][k] + A[k][j]
        Next[i][j] := Next[i][k]        ← 경로 successor 갱신
```

Next[u][v] 초기값: 간선 있으면 v, 없으면 -1. in-place 갱신 가능(공간 $O(n^2)$).

✎ **핵심** Space reduction: A_k가 A_{k-1}을 덮어써도 안전 (i=k,j=k 행·열은 불변, 나머지는 A_k 계산에만 쓰임) → $O(n^2)$ 공간, 시간은 여전히 $O(n^3)$.

행렬 진화 $A^0 \to A^5$ (전체 과정) — 초기 A^0(직접 간선) → 중간점을 하나씩 허용하며 갱신 → 최종 A^5(모든 중간점 허용 = 정답).

A^0	1	2	3	4	5
1	0	10	3	20	–
2	–	0	–	5	–
3	–	2	0	–	15
4	–	–	–	0	11
5	–	–	–	–	0

A^0 — 중간점 없음(직접 간선)

A^5	1	2	3	4	5
1	0	5	3	10	18
2	–	0	–	5	16
3	–	2	0	7	15
4	–	–	–	0	11
5	–	–	–	–	0

A^5 — 최종 최단거리(정답)

경로 복원 — Next chain — 거리 행렬만으론 실제 경로 모름 → `Next` 테이블을 따라 hop.

Path retrieval (1→4)

```
path := [u]
while u ≠ v do
  u := Next[u][v]
  path.append(u)
u=1: Next[1,4]=3 → [1,3]
u=3: Next[3,4]=2 → [1,3,2]
u=2: Next[2,4]=4 → [1,3,2,4]  ✓ 거리 3+2+5=10 = A⁵[1,4]
```

비교 · 다른 알고리즘 p.51-55

	Dijkstra ×n	Floyd
용도	single-source 반복	all-pairs 한 번
시간	$O(n^2 \log n)$ (heap, sparse)	$O(n^3)$
유리	희소(sparse) 그래프	밀집(dense) 그래프
음수 간선	불가(정당성 붕괴)	가능 (negative cycle만 없으면)

밀집·음수 간선이면 Floyd / 희소면 Dijkstra×n

기타 최단경로 알고리즘

- **Bellman-Ford** — single-source, 음수 간선 허용, `O(n·e)` (Dijkstra보다 느림)
- **Johnson** — all-pairs, 음수 허용, `O(n²log n + n·e)` (희소면 Floyd보다↑)
- **A* search** — single-pair, *휴리스틱*으로 탐색을 좁은 cone으로 가속

★ **필수암기** Dijkstra=single-source·비음·$O(n^2)$/$O(e \log n)$ · D[v]=S내부만통과 최단 · 음수간선 실패 · Floyd=all-pairs·$O(n^3)$·DP점화식·음수가능(neg-cycle 없을때) · DeleteMin=O(log n)

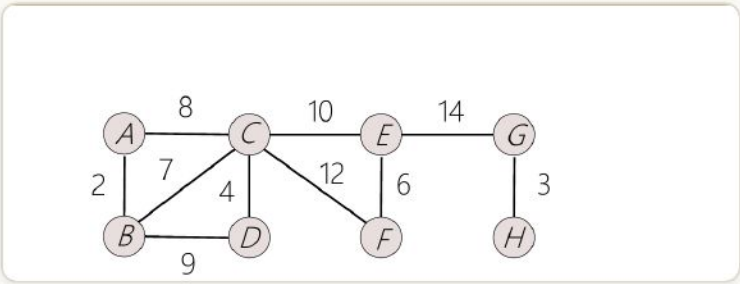
Ch 16 Minimum Spanning Trees — 최소 비용 신장 트리

가중 연결 무방향 그래프에서 모든 정점을 연결하는 최소 비용 트리 · Prim · Kruskal · Sollin

MST 정의 & Greedy 전략 3가지 p.1-8

MST (Minimum-Cost Spanning Tree) — 가중 *연결 무방향* 그래프에서 **모든 정점을 포함**하면서 **간선 비용의 합이 최소**인 트리.

- spanning tree는 항상 **n-1**개 간선 → n개 간선 중 n-1개 선택(또는 나머지 버리기) 문제
- 입력 조건: *가중* · *연결* · *무방향* — 비연결이면 spanning tree 자체 불가
- 응용: 네트워크 인프라 · 도로망 · 회로 배선 ("모두 연결 + 총비용 최소")



예제 그래프 — 8 정점 10 간선, spanning tree = 7개 간선 (p.5)

세 가지 Greedy 전략 — 모두 cut property 기반 greedy. 차이는 '어떤 단위로 간선을 고르느냐'.

- **Prim** — *1-정점 트리*에서 시작, 매 단계 **가장 싼 간선(+정점)**을 붙여 n-정점 트리로 성장
- **Kruskal** — *n-정점 forest*에서 시작, 간선을 **비용 오름차순**으로 검토, 사이클 안 만들면 채택
- **Sollin (Borůvka)** — 각 component가 *동시에* 최소 inter-cluster 간선 선택 → **병렬화** 강점

⚠ **함정** "각 정점의 가장 싼 outgoing edge"를 단순히 고르는 *순진한 greedy*는 동작하지 않는다 — 사이클 발생, 전역 최적 미보장. Prim·Kruskal은 cut property로 '안전한 선택'임이 보장된다.

Prim 알고리즘 p.9-17

3대 자료구조 `Q` , `D[k]` , `P[k]` — 우선순위 큐 기반 구현의 핵심 세 배열.

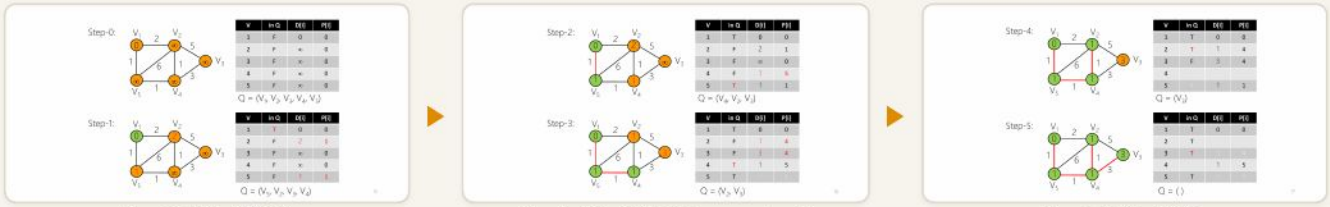
D=트리거리
P=부모
Q=미편입큐

- **Q** — 아직 트리에 포함되지 않은 정점들의 우선순위 큐
- **D[k]** — 정점 k의 **현재 트리까지 최단 거리** (= Q 안의 우선순위)
- **P[k]** — k를 트리에 잇는 부모 정점 (MST 간선 복원용)

Prim (우선순위 큐)

```
Initialize:
  Q ← V
  D[1] := 0;  P[1] := 0
  for i := 2 to n do  D[i] := ∞, P[i] := 0
while Q ≠ ∅ do
  w ← DeleteMin(Q)            ← D[.] 최소 정점 트리에 편입
  for each v in Adj[w] do
    if v ∈ Q and C[w,v] < D[v] then
      D[v] := C[w, v]         ← decrease-key
      P[v] := w
```

decrease-key: 새 트리 정점 w 덕에 바깥 정점의 D[v]가 더 싸질 때 갱신.



Step-0~1: $V_1 \rightarrow V_5$(D=1) | Step-2~3: $V_5 \rightarrow V_4$(D=1)·decrease-key V_2 | Step-4~5: $V_2 \rightarrow V_3$ 완료

Prim step-by-step — 초록=확정, 빨강=MST 간선, 오렌지=현재 큐 최솟값 (p.15-17)

D/P 전체 추적 — step마다 DeleteMin 직후 D·P 상태. 초기 D[1]=0, 나머지 ∞.

단계	D[.] 갱신 (decrease-key)	Q 상태
Init	V_1=0,V_2=∞,V_3=∞,V_4=∞,V_5=∞	모두 0
Step-1 (V_1↑)	D[V_2]:=2(P=1), D[V_5]:=1(P=1)	Q=(V_5,V_2,V_3,V_4)
Step-2 (V_5↑)	D[V_4]:=1(P=5)	Q=(V_4,V_2,V_3)
Step-3 (V_4↑)	D[V_2]:=1(P=4)↓, D[V_3]:=3(P=4)	Q=(V_2,V_3)
Step-4 (V_2↑)	갱신 없음	Q=(V_3)
Step-5 (V_3↑)	MST 완성	Q=∅

분홍=decrease-key 발생, 초록=완성. MST 간선: V_1-V_5(1)·V_5-V_4(1)·V_4-V_2(1)·V_4-V_3(3)

Prim 정당성 & 복잡도 p.18-23

MST Property (Cut Property) — 연결 그래프 G=(V,E)와 진부분집합 U⊂V에 대해, u∈U·v∈V-U를 잇는 **최소 비용 간선 (u,v)**를 포함하는 MST가 반드시 존재한다.

- 이것이 **greedy-choice property** — 국소 최적 선택이 전역 최적으로 이어짐
- Prim은 매 step 'U=트리 / V-U=바깥'이라는 cut에 이 성질을 반복 적용

귀류법 증명 — (u,v)를 포함하는 MST가 없다고 가정 → T에 (u,v) 추가 시 사이클 생성 → 사이클 안의 다른 cut 간선 (u',v') 존재

- 가정상 c(u,v) ≤ c(u',v') → (u',v')를 (u,v)로 교체 → 비용 같거나 감소한 또 다른 MST **모순**
- 따라서 (u,v)를 포함하는 MST 존재 — cut property 성립
- 음수 간선도 OK: MST는 최단 경로가 아니라 **총비용 최소**만 따짐 → 교환 논증에 영향 없음

복잡도 — 자료구조에 따라 둘 — 병목은 '*D 최소 정점 선택*'(DeleteMin) 연산.

- **Adjacency matrix + 선형 스캔**: 초기화 O(n) + while n번 × 스캔 O(n) = **O(n²)** → *dense* 그래프 유리
- **Adjacency list + min-heap**: DeleteMin n번 O(n log n) + decrease-key e번 O(e log n) = **O(e log n)** → *sparse* 그래프 유리

Kruskal 알고리즘 p.24-31

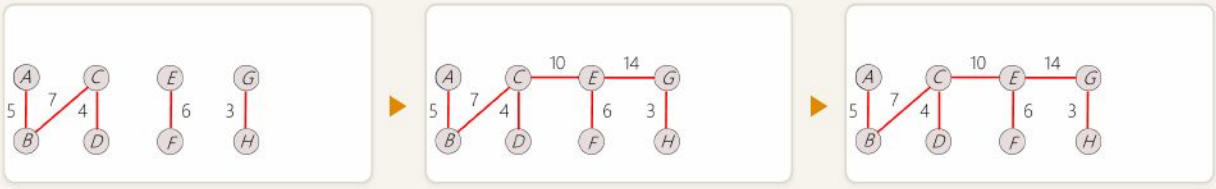
Kruskal — 간선 중심 Greedy — 간선 없는 forest(정점 n개)에서 시작. 간선을 **비용 오름차순**으로 검토 — 사이클 안 만들면 채택, 만들면 버린다.

- n-1개 간선이 모이면 MST 완성. '사이클 판정' = **Union-Find**의 FIND 결과 비교
- 채택 순서: (G,H)=3 · (C,D)=4 · (A,B)=5 · (E,F)=6 · (B,C)=7 · (A,C)=8 **거부** · (B,D)=9 **거부** · (C,E)=10 · (C,F)=12 **거부** · (E,G)=14

Kruskal

```
Initialize: forest ← n개 정점, 간선 없음
Sort edges by cost (ascending) or use min-heap
for each edge (u, v) in order do
  r ← FIND(u);  s ← FIND(v)
  if r ≠ s then              ⟵ 다른 component → 사이클 없음
    add (u, v) to MST
    UNION(r, s)
  else                       ⟵ 같은 component → 사이클 → skip
    reject (u, v)
```

핵심 연산: FIND(어느 집합?) + UNION(두 집합 병합). 효율은 Union-Find 구현에 달림.



초기 5간선 채택 (G,H)·(C,D)·(A,B)·(E,F)·(B,C) ▶ (B,D)거부·(C,E)채택·(C,F)거부·(E,G)채택 ▶ MST 완성 — 7간선, 비용=49

Kruskal step-by-step — 빨강=채택 간선, 오름차순 검토·사이클 거부 (p.24-28)

✎ **핵심** MST **유일성**: 간선 비용이 모두 distinct이면 MST는 **유일**. 교환 논증에서 동률이 없으면 매 cut의 최소 간선이 유일하게 결정되기 때문. 동률 존재 시 간선 구성이 다른 복수 MST 가능 (단 총비용은 동일).

Union-Find (Disjoint-Set) p.32-52

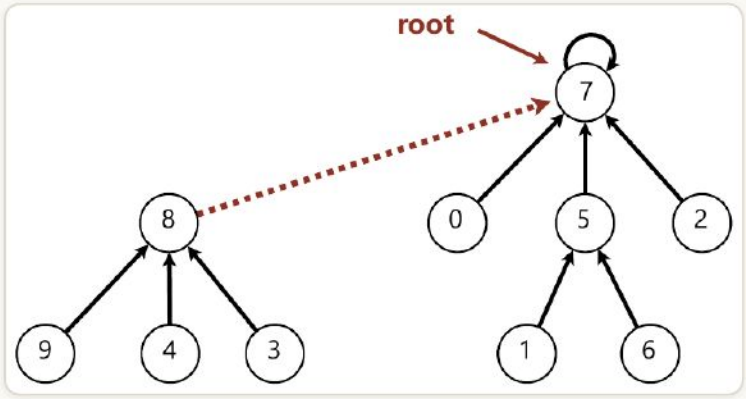
m=총 호출수
n=원소수

세 연산 — 서로소 집합 컬렉션에 대한 세 연산.

- **MAKE-SET(x)** — 원소 x만 든 새 집합 생성 (`parent[x] ← x`)
- **FIND(x)** — x가 속한 집합의 **대표원소(루트)** 반환 (parent 포인터 타고 올라감)
- **UNION(x,y)** — x·y의 집합을 합집합으로 교체 (루트 하나를 다른 루트 밑에 매달기)
- Kruskal 사이클 판정 = `FIND(u) == FIND(v)` ? 사이클 : 채택+UNION

Parent-link Representation — 각 집합을 *트리*로 표현. 루트가 대표원소(자기 자신을 가리킴). 배열 `parent[]` 로 구현.

- `parent[i] = j` ⟺ 원소 i의 부모가 j. 루트: `parent[i] = i`



UNION(3,5) — 왼쪽 트리 루트를 오른쪽 루트 7 밑에 매달기 (p.35)

Naïve Linking — 최악 Θ(n) — 루트를 그냥 다른 루트 밑에 매단다(크기·높이 무시). 최악: UNION(1,2), UNION(2,3), … → 일렬 사슬 → FIND가 **Θ(n)**.

Naïve Linking

```
MAKE-SET(x):  parent[x] ← x
FIND(x):      while x ≠ parent[x]  do  x ← parent[x];  return x
UNION(x, y):  r ← FIND(x);  s ← FIND(y);  parent[r] ← s
```

크기·높이 고려 없음 → 트리가 한쪽으로 늘어져 최악 높이 n-1 발생.

Link-by-Size / Link-by-Rank — O(log n) — 두 최적화 모두 트리 높이를 **O(log n)**으로 묶어 UNION·FIND를 O(log n)으로 만든다.

- **Link-by-size**: 각 루트에 `size[]` (노드 수) 유지 — *작은 트리를 큰 트리* 루트 밑에
- UNION: size[r]>size[s] → parent[s]←r, size[r]+=size[s]; else 반대
- **Link-by-rank**: 각 노드에 `rank[]` (높이 추정치, 초기 0) 유지 — *낮은 rank를 높은 rank* 밑에
- 동률이면 한쪽 rank += 1 (동률일 때만 높이가 1 증가하므로 rank가 정확히 높이를 추정)

Link-by-size UNION

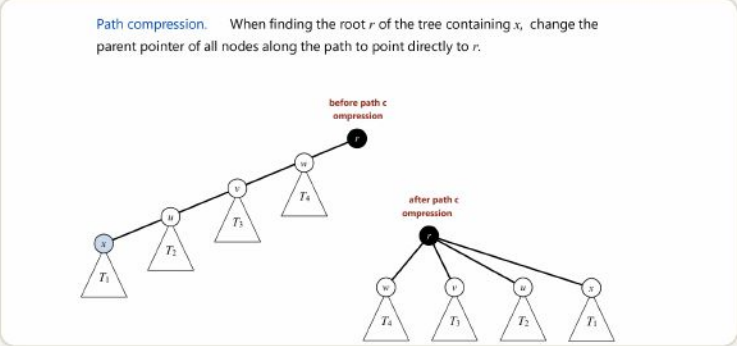
```
r ← FIND(x);  s ← FIND(y)
if r = s then return
if size[r] > size[s] then
  parent[s] ← r;  size[r] += size[s]
else
  parent[r] ← s;  size[s] += size[r]
```

Link-by-rank UNION

```
r ← FIND(x);  s ← FIND(y)
if r = s then return
if rank[r] > rank[s] then  parent[s] ← r
else if rank[r] < rank[s] then  parent[r] ← s
else
  parent[r] ← s;  rank[s] += 1
```

Path Compression — FIND(x)로 루트 r을 찾는 *김에* 경로상 **모든 노드의 parent를 r로** 직접 바꾼다. 이후 FIND가 O(1) 수준으로 빨라짐.

- 재귀 한 줄: `if x≠parent[x]: parent[x]←FIND(parent[x]); return parent[x]`
- 되돌아오는 길에 경로 노드 전부를 루트 직속으로 끌어올림 → 트리가 *납작*해짐



before/after path compression — 경로 노드 모두 루트 직속으로 (p.44)

복잡도 정리 — linking + path compression 결합 시 m번 연산이 **O(m log n)** (실질 역 Ackermann α(n) ≈ 상수).

- link-by-rank + path compression: O(m log n)
- link-by-size + path compression: O(m log n)
- **Kruskal 전체**: 간선 정렬 O(e log e) + 간선마다 FIND·UNION O(log e) × e번 = **O(e log n)**

응용 · 클러스터링 · MST vs 최단경로 p.53-65

Single-Link Clustering = Kruskal — k-clustering: 객체를 k개 그룹으로 나누되 *다른 클러스터끼리 거리 최대화*. Single-link(두 클러스터 간 거리 = 가장 가까운 쌍).

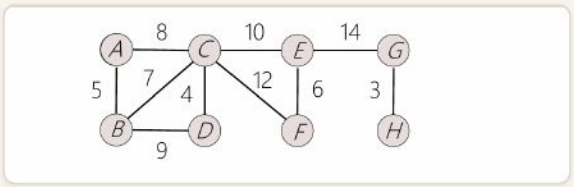
- 알고리즘: Kruskal을 실행하다 **component 수가 k개가 되면 정지** → 그게 k-clustering
- MST의 비싼 k-1개 간선을 제거하면 곧 k-clustering — dendrogram으로 시각화

⚠ **함정** **MST ≠ 최단경로 트리(Dijkstra)**. MST 목표 = *총 비용 합 최소* vs 최단경로 목표 = *source에서 각 정점까지 거리 최소*. MST 위의 두 정점 간 경로가 최단경로라는 보장은 없다.

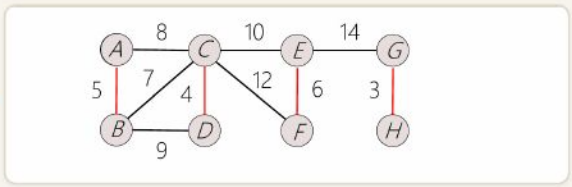
Sollin (Borůvka) 알고리즘 p.67-71

Sollin's Algorithm — n-정점 forest에서 각 component가 **동시에** 다른 component로의 최소 비용 간선 선택.

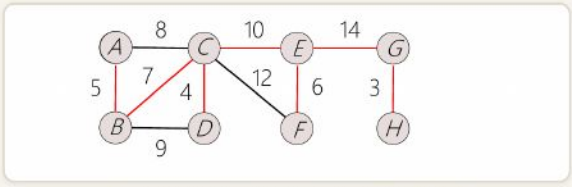
- 중복 선택·사이클 제거 → component 1개 남을 때까지 반복
- 매 라운드 component 수가 최소 절반으로 감소 → **O(log n) 라운드**
- Prim·Kruskal의 순차 진행과 달리 **병렬화**에 강점 — 각 component가 독립적



R1 선택 전: 각 정점이 최소 inter-cluster 간선 고름



R1 결과: 8→4 components ({A,B}·{C,D}·{E,F}·{G,H})



R2: {A,B}:7·{C,D}:7·{E,F}:10·{G,H}:14 → MST 완성

Sollin 라운드별 병합 — 동시에 최소 간선 선택, 중복 제거 (p.69-71)

Sollin 예제 (p.69-71) — 라운드 1: 각 정점이 자신의 최소 inter-cluster 간선 선택 → 쌍으로 겹쳐 4개 클러스터. 라운드 2: 4→1개.

- R1 선택: A=5·B=5 → {A,B}, C=4·D=4 → {C,D}, E=6·F=6 → {E,F}, G=3·H=3 → {G,H}
- R2 선택: {A,B}=7(B-C), {C,D}=7(B-C), {E,F}=10(C-E), {G,H}=14(E-G) → 1개

Prim vs Kruskal vs Sollin — 비교 p.66-74

	Prim	Kruskal	Sollin
시작	1-정점 트리	n-정점 forest	n-정점 forest
진행 단위	정점 1개씩 추가	간선 1개씩 추가	component 전체 동시
자료구조	priority queue	Union-Find	Union-Find
복잡도	O(n²)/O(e log n)	O(e log n)	O(e log n)
dense	유리 (O(V log V))	보통	보통
sparse	보통	유리 (단순 구조)	보통
병렬화	불리	보통	강점
구현	어려움	쉬움	중간

dense→Prim(Fibonacci heap), sparse·구현 단순→Kruskal, 병렬→Sollin

✎ **핵심** 세 알고리즘 모두 **cut property** 기반 greedy이므로 **같은 최소 비용**의 MST를 만든다. 간선 비용 동률이 있으면 간선 구성이 다른 복수 MST 가능(총비용은 동일). Euclidean MST: brute-force O(N²) → *Delaunay 삼각분할* 이용 시 O(N log N).

★ **필수암기** MST=가중연결무방향·n-1간선·총비용최소 · Prim=정점확장·D[.]+P[.]+Q·O(n²)/O(e log n) · cut property=최소비용cross-cut간선⊆MST·귀류법 · 음수간선=Prim도OK(MST는총비용) · Kruskal=간선오름차순·사이클skip·O(e log n) · Union-Find: naive=Θ(n)·by-rank/size=O(log n)·path-compression+linking=O(m log n) · Sollin=병렬·O(log n)라운드 · single-link clustering=Kruskal k정지

Ch 17 String — 문자열 인덱싱

Trie → Compressed Trie → Suffix Tree → Exact Matching → Generalized → Suffix Array

String · Trie · Compressed Trie p.1-7

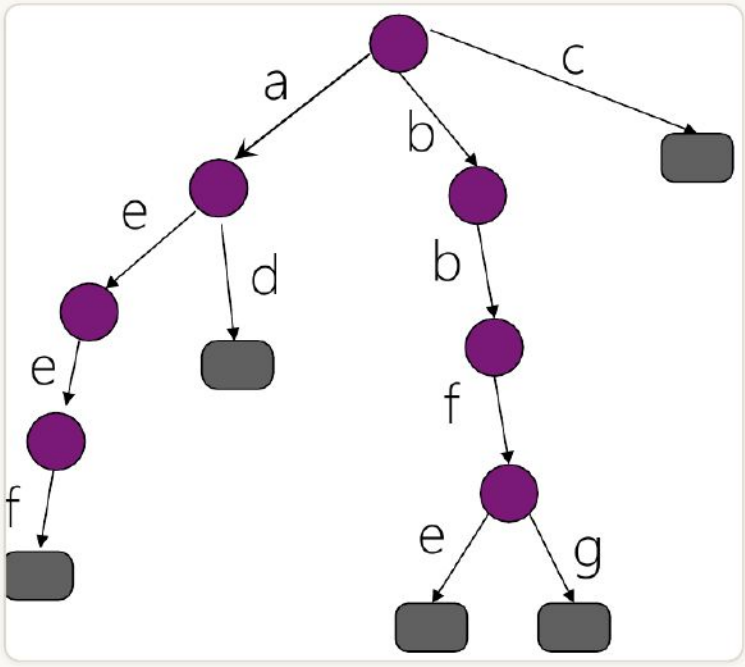
String (문자열) 정의 — 유한 알파벳 위 문자들의 **순서 있는 나열** (인덱스 1…m). 순서가 있어야 substring·prefix·suffix가 정합적으로 정의됨.

- 도메인 독립: 영어·DNA(*GATC*)·코드 — 알파벳 크기만 다를 뿐 *같은 알고리즘* 적용
- **char**: C=8b ASCII, Java=16b Unicode, Python=내부 Unicode(인코딩 은닉)
- Unicode — 코드 포인트(code point) vs 바이트 표현(UTF-8/16/32) 구분 필수

Trie — 문자열 *집합* 위의 트리 — 예: {aeef, ad, bbfe, bbfg, c}. **접두사 공유**로 O(L) 검색.

O(L) 검색
집합 크기 무관

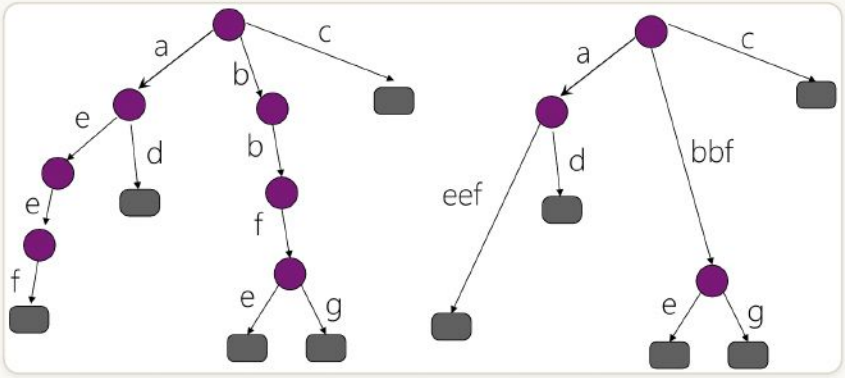
- 전제: 집합 내 어떤 문자열도 다른 것의 **~~접두사(prefix)가 아님~~** — 위반 시 leaf:1 대응 붕괴
- 간선 라벨 = 한 글자, *같은 노드의 두 간선은 동일 라벨 불가*



Trie 예제: {aeef, ad, bbfe, bbfg, c} (p.6)

Compressed Trie — 자식이 *하나뿐인* 노드 사슬을 합치고 라벨을 부분문자열로 연결. 노드 수·깊이 감소, 점근 복잡도 동일.

- 간선 저장 = `(start, end)` 포인터 — 부분문자열 중복 없이 표현
- 분기 판단: "어느 자식?" → **"어느 자식, 라벨 안 어디까지?"**로 확장



Compressed Trie: 사슬 합치기 전후 비교 (p.7)

Suffix Tree — 정의 p.8-11

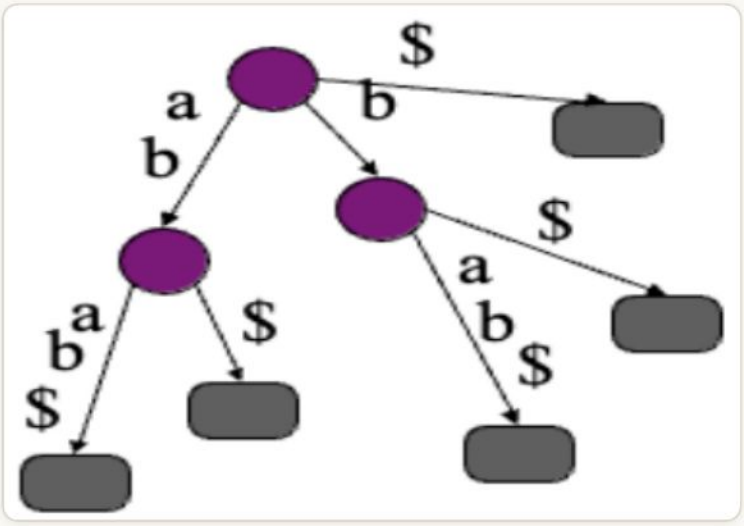
Suffix Tree — 텍스트 T의 **모든 접미사** 위 Compressed Trie. 한 번 구축 → 이후 패턴 질의가 텍스트 크기와 무관.

m=텍스트 길이
leaf=접미사 위치

- 정확히 **m개 leaf**, 각 내부 노드(root 제외) 자식 ≥ 2
- 각 간선 라벨 = T의 *비어있지 않은 부분 문자열*
- **첫 글자 규칙**: 같은 노드에서 두 간선은 *같은 첫 글자 불가* → 분기가 *결정적*
- leaf i의 root→leaf 경로를 이으면 정확히 T[i,⋯,m]을 출력

센티널 $ — T 끝에 알파벳에 없는 특수 문자 `$` 추가 → 어떤 접미사도 다른 접미사의 **접두사가 되지 않음** 보장 → 모든 접미사가 leaf에서 끝남.

- 압축 후에도 **O(m) 노드**만 유지 → 실용적 인덱스 가능



Suffix Tree(T=abab$) 완성 구조 — leaf 번호=시작 위치 (p.11)

Naive Construction — 구축 알고리즘 p.12-16

순차 삽입 (Naive) — T=abab$. 접미사를 *긴 것부터* 하나씩 삽입 — 기존 트리에서 공통 접두사 경로를 따라가다 분기.

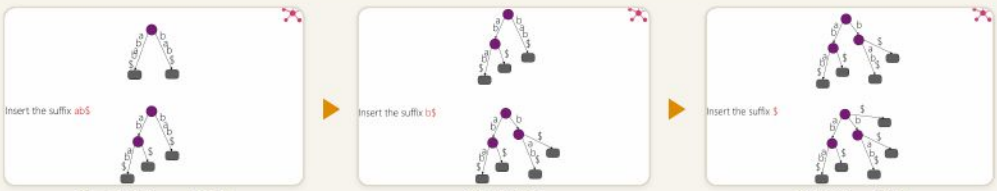
삽입당 O(m)
총 O(m²)

- 삽입 1개: O(m) 경로 걷기 × m회 = **O(m²)** 총 비용
- 공간은 공유 덕분에 **O(m) 노드** 유지

Naive Suffix Tree 구축

```
Insert T[1..m]$ into empty tree    ← 전체 문자열(가장 긴 접미사)
for i := 2 to m+1 do
  walk from root consuming T[i..m]$
  if mismatch mid-edge then
    split edge, create new internal node
  create new leaf for suffix T[i..m]$
```

O(m) 삽입 × m회 = O(m²). Weiner·McCreight·Ukkonen은 suffix link로 O(m) 달성.



① ab$ 삽입 — ab 공유 ② b$ 삽입 ③ $ 삽입 → 완성

T=abab$ Naive 구축 — 긴 접미사(abab$·bab$)부터 차례로 삽입 (p.15-17)

✎ **핵심** 슬라이드 leaf 번호: 1=abab$, 2=bab$, 3=ab$, 4=b$, 5=$. 구축 후 **leaf i ↔ 시작위치 i** 대응 확인이 정의 검증의 핵심.

Exact String Matching p.17-21

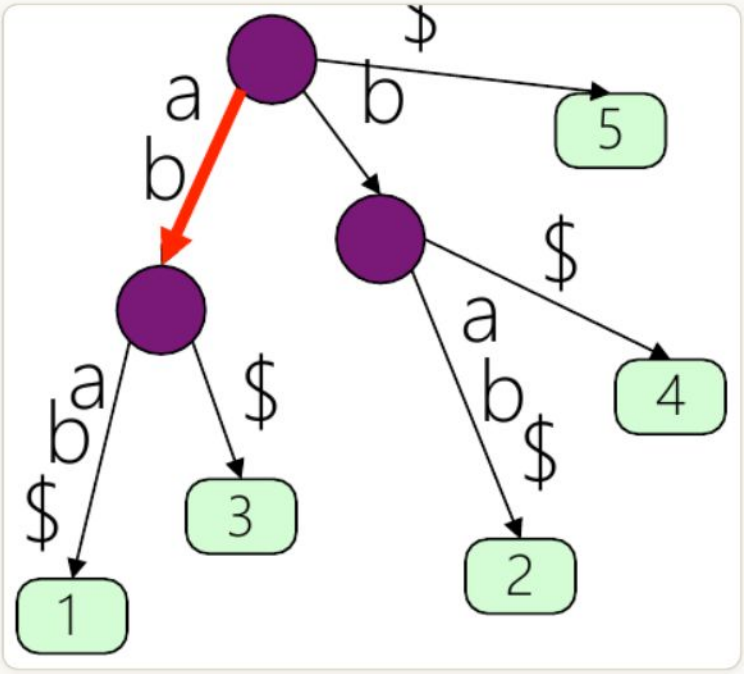
Suffix Tree 기반 매칭 — T에 대한 suffix tree를 미리 구축 → 패턴 s(|s|=n)를 root에서 **트리 경로를 따라 내려가며** 소비.

패턴 소비 후 서브트리 수집

- s 소진 성공 → 도달 노드의 **서브트리 leaf 전부** = 모든 출현 위치 k개
- 다음 글자에 해당하는 간선 없음 → *출현 없음*

~~시간 복잡도 O(m+n+k)~~ — 세 단계 분해:

- **구축**: O(m) — Weiner·McCreight·Ukkonen 선형 알고리즘
- **매칭**: O(n), n=|패턴| — 경로 내려가기
- **열거**: O(k), k=출현 횟수 — leaf 수집
- 분할상환 우위: **m은 일회성 전처리**, 반복 질의는 n+k만 지불



aba ⊂ abab$? — root→a→b 경로 후 서브트리 leaf={1,3} (p.21-22)

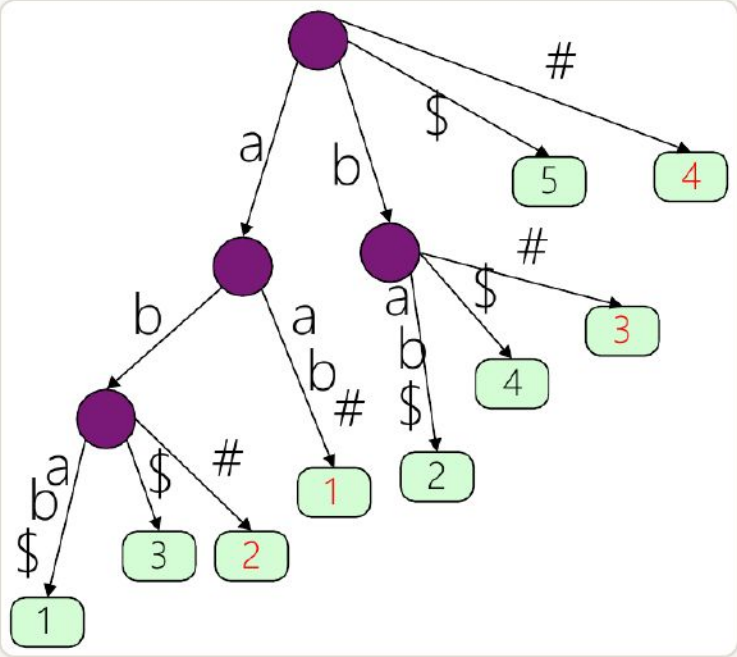
⚠ **함정** Naive 구축 O(m²) vs 선형 구축 O(m): *결과 트리는 동일.* naive는 개념 이해용, 실무는 O(m) 알고리즘 사용.

Generalized Suffix Tree — 다중 문자열 p.22-24

Generalized Suffix Tree — 문자열 컬렉션 T={$t_1,\cdots,t_k$}의 *모든 접미사 합집합* 위 단일 compressed trie.

컬렉션 전체 단일 트리

- 각 t_i마다 **서로 다른 센티널** 부착 (예: $, #) → leaf 출처 구분
- leaf = (t_i, position) 쌍으로 *정체성 모호 없이* 유지
- 공간: 모든 입력 크기 합에 *선형* — 상수 인자 큰 것이 단점



Generalized suffix tree: t_1=abab, t_2=aab — $ / # 센티널로 구분 (p.27)

Suffix Array — 경량 대안 p.25-33

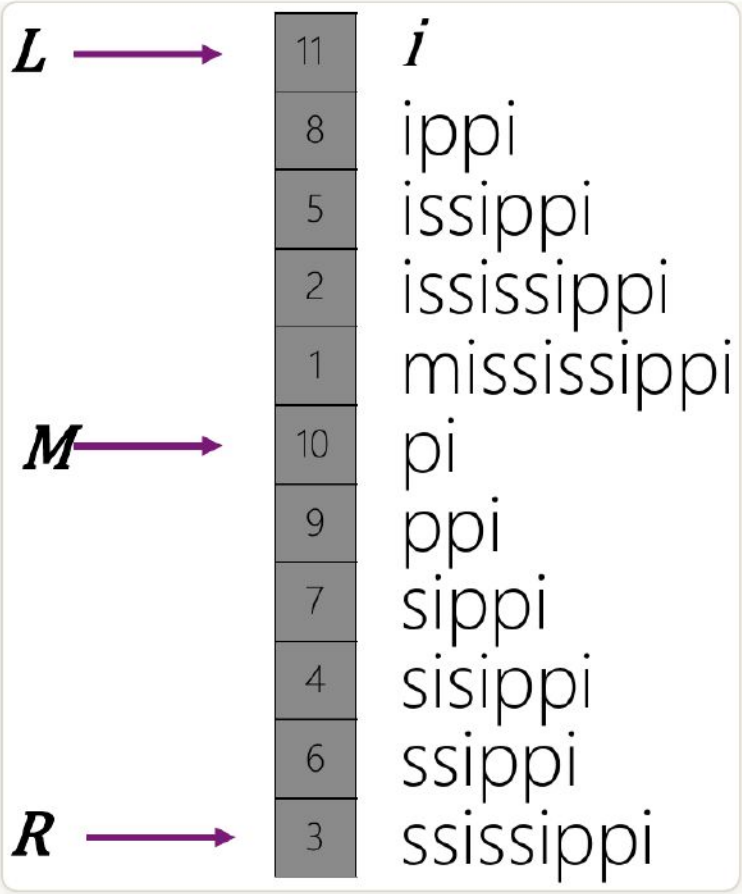
Suffix Array 정의 — 모든 접미사의 시작 위치를 **사전식(lexicographic) 순서**로 정렬해 저장한 정수 배열.

정수 배열
= 사전순 위치

- T=abab → 정렬 순서: ab < abab < b < bab → 배열 **[3, 1, 4, 2]**
- 노드 대신 *위치당 정수 하나*만 저장 → suffix tree 대비 대폭 경량
- 배열 원소의 정렬 순서가 접두사 공유 정보를 *암시적으로* 인코딩

검색 — 이진 탐색 — 패턴 s의 모든 출현은 정렬된 배열에서 **연속 구간**으로 모임 → 이진 탐색으로 구간 확정.

- s 이상인 첫 위치 L, s 초과인 첫 위치 R 탐색 → [L,R) 구간
- 시간: **O(n log m)** — 문자 비교 O(n) × 이진탐색 log m회
- Suffix tree O(n+k) 대비 느리지만 *캐시 친화적·직렬화 용이*



mississippi suffix array: s=issa → L=index of issippi, R=sippi (p.26)

Suffix Array 구축 방법

- **방법 1 (트리 경유)**: suffix tree DFS로 leaf 수집 → `O(m)` (트리 구조 먼저 필요)
- **방법 2 (직접 정렬)**: 정렬 기반 → `O(m·r·log m)`, r=최장 반복 길이
- 현대 라이브러리(SA-IS, divsufsort): `O(m)` 직접 구성

비교 · 정리 p.31-33

구조	전처리	질의	공간	특징
Trie	O(총 길이)	O(L)	O(총 글자)	집합·접두사 공유
Compressed Trie	O(총 길이)	O(L)	O(문자열 수)	사슬 합치기
Suffix Tree	O(m) *선형	O(n+k)	O(m) *상수 큼	단일 텍스트 인덱스
Suffix Array	O(m)	O(n log m)	O(m) *경량	이진탐색, 캐시↑

*선형 구축은 Weiner/McCreight/Ukkonen. n=패턴 길이, k=출현 횟수, m=텍스트 길이

✎ **핵심** 선택 기준: 시간 우선 → Suffix Tree O(n+k) / 공간 우선 → Suffix Array O(n log m) / 다중 텍스트 → Generalized suffix tree (서로 다른 센티널 필수)

★ **필수암기** Trie=집합 트리·접두사없음 전제·O(L) / CompressedTrie=사슬압축·(start,end) 포인터 / SuffixTree=T 전체 접미사 compressed trie·sentinel $·m leaf·O(m) 구축·O(n+k) 매칭 / NaiveO(m²) / Exact match=트리 따라가다 서브트리 leaf / SuffixArray=[정수배열·사전순]·이진탐색·O(n log m)